LAB 6 · LECTURE 6

Lecture 6 Lab: Chunking a Real PDF

Vector Databases Course | Estimated time: 75 minutes | Platform: Google Colab (free)

Goal

You will download a real research paper as a PDF, inspect and clean the extracted text, and cut it into chunks four different ways. You will then attach metadata and stable IDs, and measure which strategy retrieves the right passage most often.

- Extract and clean text from a PDF, and see what PDF extraction gets wrong.
- Implement fixed-size, recursive, sentence-window and semantic chunking.
- Check chunk lengths in tokens against the model's input limit.
- Build chunk records with deterministic IDs, content hashes and page metadata.
- Evaluate strategies and sizes with a hit rate at 3 on eight questions.

Setup

Create a new Colab notebook and run each block in its own cell, in order.

Cell 1: install and import

```
!pip install -q pypdf requests sentence-transformers numpy pandas matplotlib

import re, json, hashlib
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
import requests
from pypdf import PdfReader
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")
```

Part 1: Get the text out of a PDF

We use the 2017 paper 'Attention Is All You Need' from arXiv. If the download is blocked, upload any text-heavy PDF named paper.pdf, and adapt the questions in Cell 9.

Cell 2: download and extract

```
url = "https://arxiv.org/pdf/1706.03762"
r = requests.get(url, headers={"User-Agent": "Mozilla/5.0"}, timeout=60)
open("paper.pdf", "wb").write(r.content)

reader = PdfReader("paper.pdf")
pages = [p.extract_text() or "" for p in reader.pages]
print(len(pages), "pages,", sum(len(p) for p in pages), "characters")
print(pages[2][:700])
```

YOUR TURN Read the extracted text. List three things that would hurt retrieval (for example broken words, repeated headers, equations or table noise).

Cell 3: clean the text

```
def clean(text):
    text = re.sub(r"-\n(?=[a-z])", "", text)       # rejoin hyphenated line breaks
    text = re.sub(r"(?m)^\s*\d+\s*$", "", text)     # drop lines with only a number
    text = re.sub(r"[ \t]+", " ", text)
    return text.strip()

doc_text = "\n".join(clean(p) for p in pages)
flat = re.sub(r"\s+", " ", doc_text)               # one long line for sentence tools
print(len(doc_text), len(flat))
```

Part 2: Four chunkers

Cell 4: fixed size with overlap, and recursive

```
def fixed_chunks(text, size=800, overlap=150):
    step = size - overlap
    return [text[i:i + size] for i in range(0, max(len(text) - overlap, 1), step)]

def recursive_chunks(text, size=800, seps=("\n\n", "\n", ". ", " ")):
    if len(text) <= size:
        return [text]
    for k, sep in enumerate(seps):
        if sep in text:
            parts = [p + sep for p in text.split(sep)]
            parts[-1] = parts[-1][:-len(sep)]
            rest = seps[k + 1:]
            break
    else:
        return [text[i:i + size] for i in range(0, len(text), size)]
    chunks, cur = [], ""
    for part in parts:
        if len(part) > size:
            if cur:
                chunks.append(cur); cur = ""
            chunks.extend(recursive_chunks(part, size, rest))
        elif len(cur) + len(part) <= size:
            cur += part
        else:
            chunks.append(cur); cur = part
    if cur:
        chunks.append(cur)
    return [c.strip() for c in chunks if c.strip()]
```

Cell 5: sentence-window and semantic

```
def sentence_chunks(text, n=5, overlap=1):
    sents = re.split(r"(?<=[.!?])\s+", text)
    return [" ".join(sents[i:i + n]) for i in range(0, len(sents), n - overlap)]

def semantic_chunks(text, percentile=20, max_chars=1200):
    sents = [s for s in re.split(r"(?<=[.!?])\s+", text) if len(s) > 20]
    E = model.encode(sents, normalize_embeddings=True)
    sims = np.sum(E[:-1] * E[1:], axis=1)            # neighbour similarity
    cut = np.percentile(sims, percentile)
    chunks, cur = [], sents[0]
    for s, sim in zip(sents[1:], sims):
        if sim < cut or len(cur) + len(s) > max_chars:
            chunks.append(cur); cur = s
        else:
            cur += " " + s
    chunks.append(cur)
    return chunks
```

Cell 6: run all four and compare

```
strategies = {
    "fixed":     fixed_chunks(flat),
    "recursive": recursive_chunks(doc_text),
    "sentence":  sentence_chunks(flat),
    "semantic":  semantic_chunks(flat),
}

def stats(chunks):
    lens = np.array([len(c) for c in chunks])
    ends = np.mean([c.rstrip()[-1:] in ".?!" for c in chunks])
    return {"chunks": len(chunks), "mean": int(lens.mean()), "min": int(lens.min()),
            "max": int(lens.max()), "end at sentence": round(float(ends), 2)}

pd.DataFrame({k: stats(v) for k, v in strategies.items()}).T
```

YOUR TURN Which strategy produces the most uniform chunk sizes, and which most often ends at a sentence boundary? Print three chunks from each strategy and describe one weakness you can see.

Cell 7: token lengths versus the model limit

```
limit = model.max_seq_length
print("model limit:", limit, "tokens")
for name, ch in strategies.items():
    toks = [len(model.tokenizer(c)["input_ids"]) for c in ch]
    print(f"{name:10s} over limit: {sum(t > limit for t in toks):3d} of {len(ch)}"
          f"   max {max(toks)} tokens")
```

YOUR TURN Which chunks would be silently truncated by the model? What setting would you change for the fixed chunker so none exceed the limit? Check your answer by re-running.

Part 3: Metadata and stable IDs

Cell 8: build chunk records page by page

```
doc_id = "attention-2017"
records = []
for pno, ptext in enumerate(pages, start=1):
    for j, c in enumerate(recursive_chunks(clean(ptext), size=800)):
        records.append({
            "id": f"{doc_id}:p{pno}:{j}",
            "text": c,
            "metadata": {"source": "paper.pdf", "page": pno, "chunk": j,
                         "sha1": hashlib.sha1(c.encode()).hexdigest()[:12]},
        })

print(len(records), "records")
print(json.dumps(records[10], indent=2)[:500])
with open("chunks.jsonl", "w") as f:
    for rec in records:
        f.write(json.dumps(rec) + "\n")
```

YOUR TURN Why include the content hash and the page number? Explain how the hash lets you skip unchanged pages on the next ingestion run, and what you would do for a page whose hash changed.

Part 4: Which strategy retrieves best?

Each question comes with a key phrase that appears in the answer passage. A strategy gets a hit when one of the top three chunks contains the phrase. Hyphens and whitespace are ignored when matching, because PDF extraction breaks words unpredictably.

Cell 9: evaluation set and hit rate

```
def norm(s):
    return re.sub(r"[\s\-]", "", s.lower())

eval_set = [
    ("How is attention computed from queries, keys and values?",
     "scaled dot-product attention"),
    ("Why does the model use several attention heads?",
     "multi-head attention"),
    ("How does the model know word order?",
     "sinusoid"),
    ("Which optimizer was used for training?",
     "adam"),
    ("How many identical layers does the encoder have?",
     "identical layers"),
    ("Which data was used for English-German translation?",
     "wmt 2014"),
    ("What hardware was the model trained on?",
     "p100"),
    ("What BLEU score did the big model reach on English-to-German?",
     "28.4"),
]
print("answers present in text:", [norm(k) in norm(flat) for _, k in eval_set])

def hit_rate(chunks, k=3):
    E = model.encode(chunks, normalize_embeddings=True)
    hits = 0
    for q, key in eval_set:
        qv = model.encode(q, normalize_embeddings=True)
        top = np.argsort(-(E @ qv))[:k]
        hits += any(norm(key) in norm(chunks[i]) for i in top)
    return hits / len(eval_set)

scores = {name: round(hit_rate(ch), 2) for name, ch in strategies.items()}
print(scores)
pd.Series(scores).plot.bar(ylim=(0, 1), title="hit rate @ 3"); plt.show()
```

YOUR TURN If any value in the 'answers present' list is False, fix the key phrase before trusting the result. Then report the hit rate per strategy and the question each strategy misses.

Strategy	Chunks	Hit rate @ 3	A question it misses
fixed			
recursive			
sentence			
semantic			

Part 5: Sweep size and overlap

Cell 10: grid over chunk size and overlap

```
rows = []
for size in [300, 600, 1200]:
    for ov in [0.0, 0.15]:
        ch = fixed_chunks(flat, size=size, overlap=int(size * ov))
        rows.append({"size": size, "overlap": ov, "chunks": len(ch),
                     "hit@3": round(hit_rate(ch), 2)})
pd.DataFrame(rows).pivot(index="size", columns="overlap", values="hit@3")
```

YOUR TURN Which size and overlap work best? With only eight questions, how confident are you? Write two new questions in the same format and see whether the ranking changes.

Part 6: Inspect the worst chunks

Cell 11: find noisy chunks

```
def noise(c):
    letters = sum(ch.isalpha() for ch in c)
    return 1 - letters / max(len(c), 1)

ranked = sorted(strategies["recursive"], key=noise, reverse=True)
for c in ranked[:3]:
    print(round(noise(c), 2), "|", c[:200].replace("\n", " "), "\n")
```

YOUR TURN What do the noisiest chunks contain (tables, equations, references)? Decide whether to drop, clean or keep them, and write the rule you would add to clean().

Reflection questions

- Which chunk size gave the best hit rate, and why might it differ for a legal contract or a FAQ?
- Semantic chunking costs an embedding per sentence. Under what conditions would you pay for it?
- A document is edited and re-uploaded. Describe exactly how your IDs and hashes prevent duplicates.
- Your evaluation uses eight questions written by you. List two ways this could mislead you.

Deliverable

Submit the executed notebook, the completed table, chunks.jsonl, the answers to each YOUR TURN box, and short answers to the reflection questions.

Troubleshooting

- **Download fails or returns HTML:** upload your own PDF as paper.pdf and replace the questions in Cell 9.
- **Extracted text looks garbled:** the PDF may be scanned; use OCR first.
- **All hit rates are zero or the run is slow:** check the 'answers present' list, and reduce the text with pages[:6] on slow CPUs.